

Submission Worksheet

IT202-450-M2026 / module-02-worksheet-php-challenge-scenarios

Submission Data

Course	IT202-450-M2026
Assignment	Module 02 Worksheet: PHP Challenge Scenarios
Student	Shakir A. (sa3327)
Status	submitted
Worksheet Progress	100%
Potential Grade	NaN/NaN (NaN%)
Started	6/15/2026, 5:57:49 PM
Updated	6/15/2026, 7:24:32 PM
Grading Link	https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/module-02-worksheet-php-challenge-scenarios/grading/sa3327
View Link	https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/module-02-worksheet-php-challenge-scenarios/sa3327

Instructions

Complete the Module 2 PHP challenge scenarios in the homework branch, then collect focused evidence for each scenario. It's best to solve and test the PHP files first, then fill out the worksheet; however, you may also collect evidence as each scenario is solved.

Assignment Setup

- Starter reference: [Module 02 Homework starter folder](#)
- Required branch: `Module02-Homework`
- Required homework folder: `public_html/m02/hw/`
- Required starting branch: `qa`
- Starter files to copy: `base.php`, `index.php`, `scenario1.php`, `scenario2.php`, `scenario3.php`, and `scenario4.php` (both `index.php` files should be included in their proper folders)
- Recommended order:
 - Read the whole worksheet
 - Copy the starter files; commit the baseline
 - Add planning comments, commit those comments
 - Solve each scenario, test through `qa`, then collect the worksheet evidence.

Git Workflow Checklist

1. Check out `qa`.
2. Pull the latest `qa` branch with `git pull origin qa`.
3. Create the homework branch with `git checkout -b Module02-Homework`.
4. Copy the starter files into the `m02/hw`.
5. Commit the baseline starter files before solving the scenarios.
6. Push the branch with `git push origin Module02-Homework`.
7. Open a pull request from `Module02-Homework` into `qa` and keep it open while you work.

5. Commit the baseline starter files before solving the scenarios.
6. Push the branch with `git push origin Module02-Homework`.
7. Open a pull request from `Module02-Homework` into `qa` and keep it open while you work.
8. Add planning comments before the final code for every scenario.
9. Commit planning comments before the finished solution for each scenario.
10. Commit working code in small pieces as each scenario starts working.
11. After all scenarios pass locally, merge the homework pull request into `qa`.
12. Gather evidence from the deployed `qa` site and fill out the worksheet.
13. Export the worksheet PDF from the Learn Courses Platform.
14. Save the PDF in the repository under `docs/module02/`
15. Add, commit, and push the PDF to the `Module02-Homework` branch.

- `git add .`
- `git commit -m "Add module 2 worksheet PDF"`
- `git push origin Module02-Homework`

16. Upload the same PDF to Canvas.
17. Create and merge a pull request from `qa` into `prod` so the anticipated production URLs become live.
18. Switch back to `qa` with `git checkout qa`.
19. Pull the latest `qa` branch with `git pull origin qa`.

Shared Requirements

Apply these requirements to each scenario:

- Only edit code where the starter comments say edits are allowed.
- Add a comment with your UCID, date, and a brief summary of the solution approach.
- Add planning comments before the solution code.
- Commit the planning-comment stage before the final solution.
- Test the file through the supported server path before collecting output evidence. For deployed evidence, use the working `qa` URL after the site has deployed.
- Use your own code, output, branch, pull request, and deployed URLs for evidence.

Scenario Evidence Format

Each scenario uses the same evidence pattern:

- Images:
 - Show the relevant code and the matching browser output.
 - Make sure the UCID/date comment is visible in the code evidence.
 - One combined screenshot is acceptable if the code and output are both readable.
 - Use two screenshots when one image would be hard to read.
- URLs:
 - Provide the direct GitHub file link from the homework branch.
 - Provide the anticipated production URL for the PHP file.
 - Capture the working `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
 - Each file URL must end in `.php`; zero credit for that URL entry if it does not.
- Response:
 - Explain the logic in your own words.
 - Describe the steps or decisions your code makes.
 - Do not restate the prompt or list the requirements.

Submission Check

- Confirm the homework branch has been pushed.
- Confirm the pull request into `qa` exists.

- Confirm the homework branch has been pushed.
- Confirm the pull request into **qa** exists.
- Confirm the baseline, planning, and solution commits are visible.
- Confirm the **qa** version was reviewed before promotion.
- Confirm the anticipated production URLs use the same path as the tested **qa** URLs with **-qa** changed to **-prod**.
- Confirm the worksheet PDF is committed under **docs/module02/** on the homework branch.
- Upload the same worksheet PDF to Canvas.

Submission Progress

100%

Section #1: (2 pts.) Scenario 1 Odds

100%

- File to edit and test: **public_html/m02/hw/scenario1.php**
- Goal: edit the **printOdds()** function so it outputs only odd values from each array.
- Expected behavior: print odd values on one line, separated by commas and a space.
- Output restriction: there should be no leading comma and no trailing comma.
- Workflow:
 - Add/commit the baseline code
 - Add UCID/date and planning comments before the solution code.
 - Commit the planning comments before committing the finished solution.
 - Test the PHP file locally before collecting evidence on **qa**.

Task #1 (2 pts.) – Scenario 1 Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show the relevant code with the UCID/date comment visible.
- Show the full browser output after running the file.

solution with comments

browser output

Part 2: URLs 100%

Details

- Paste the direct GitHub file link from the homework branch.
 - The GitHub URL must end in **.php**; zero credit for this URL if it does not.
- Paste the anticipated production URL for the PHP file.
 - Capture the tested **qa** URL, change **-qa** to **-prod**, and keep the same path and filename.
 - The production URL must end in **.php**; zero credit for this URL if it does not.

URL #1

https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Module02-Homework/public_html/m02/hw/scenario1.php 🍀

https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Module02-Homework/public_html/m02/hw/scenario1.php 🍏

URL #2

<https://sa3327-it202-450-m2026-prod.onrender.com/m02/hw/scenario1.php> 🍏

🔗 Part 3: Response 100%

📋 Details

- Explain how your solution decides which values are odd.
- Explain how your solution avoids leading and trailing commas.
- Do not restate the prompt.

Response

The function loops through each of the values in the array and uses modulus to check if the remainder is not zero. Odd numbers do not have a remainder of zero, so I print that value + a comma. I was not sure how to remove trailing commas but leading ones are automatically not printed.

Supports **bold**, *italic*, [links](url), `code`, etc.

285 chars

Section #2: (2 pts.) Scenario 2 Sum

100%

- File to edit and test: `public_html/m02/hw/scenario2.php`
- Goal: edit the `sumValues()` function so it sums the array values and formats the result with exactly two decimal places.
- Expected behavior: store the raw sum in `total`, then store the display-ready value in `modifiedTotal`.
- Output detail: values such as `0.1`, `1`, and `0.011` should display like `0.10`, `1.00`, and `0.01`.
- Workflow:
 - Add/commit the baseline code
 - Add UCID/date and planning comments before the solution code.
 - Commit the planning comments before committing the finished solution.
 - Test the PHP file locally before collecting evidence on `qa`.

🔗 Task #1 (2 pts.) – Scenario 2 Evidence

100%

🔗 Combo #1

📋 Part 1: Images 100%

📋 Details

- Show the relevant code with the UCID/date comment visible.
- Show the full browser output after running the file.

```
1 <!-- UCID: 1234567890, Date: 2024-01-01 -->
2 <?php
3     // Array of values
4     $values = [0.1, 1, 0.011];
5
6     // Function to sum values and format to two decimal places
7     function sumValues($values) {
8         $total = 0;
9         foreach ($values as $value) {
10             $total += $value;
11         }
12         $modifiedTotal = round($total, 2);
13         return $modifiedTotal;
14     }
15
16     // Call the function and display the result
17     $result = sumValues($values);
18     echo $result;
19
20 <?php
21 // Array of values
22 $values = [0.1, 1, 0.011];
23
24 // Function to sum values and format to two decimal places
25 function sumValues($values) {
26     $total = 0;
27     foreach ($values as $value) {
28         $total += $value;
29     }
30     $modifiedTotal = round($total, 2);
31     return $modifiedTotal;
32 }
33
34 // Call the function and display the result
35 $result = sumValues($values);
36 echo $result;
```

scenario 2 code

```
1 <!-- UCID: 1234567890, Date: 2024-01-01 -->
2 <?php
3     // Array of values
4     $values = [0.1, 1, 0.011];
5
6     // Function to sum values and format to two decimal places
7     function sumValues($values) {
8         $total = 0;
9         foreach ($values as $value) {
10             $total += $value;
11         }
12         $modifiedTotal = round($total, 2);
13         return $modifiedTotal;
14     }
15
16     // Call the function and display the result
17     $result = sumValues($values);
18     echo $result;
19
20 <?php
21 // Array of values
22 $values = [0.1, 1, 0.011];
23
24 // Function to sum values and format to two decimal places
25 function sumValues($values) {
26     $total = 0;
27     foreach ($values as $value) {
28         $total += $value;
29     }
30     $modifiedTotal = round($total, 2);
31     return $modifiedTotal;
32 }
33
34 // Call the function and display the result
35 $result = sumValues($values);
36 echo $result;
```

scenario 2 browser output

🔗 Part 2: URLs 100%

📋 Details

Part 2: URLs 100%

Details

- Paste the direct GitHub file link from the homework branch.
 - The GitHub URL must end in `.php`; zero credit for this URL if it does not.
- Paste the anticipated production URL for the PHP file.
 - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
 - The production URL must end in `.php`; zero credit for this URL if it does not.

URL #1

https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Module02-Homework/public_html/m02/hw/scenario2.php 🍏

URL #2

<https://sa3327-it202-450-m2026-prod.onrender.com/m02/hw/scenario2.php> 🍏

Part 3: Response 100%

Details

- Explain how your solution sums the values.
- Explain where and how your code formats the final display value.
- Describe any floating-point math behavior you noticed.
- Do not restate the prompt.

Response

I just looped through the array and added the value to the total each time. The final display value is formatted using the built in `number_format` function.

Supports **bold**, *italic*, [links](url), ``code``, etc.

155 chars

Section #3: (2 pts.) Scenario 3 Conversion

100%

- File to edit and test: `public_html/m02/hw/scenario3.php`
- Goal: edit the `bePositive()` function so each value becomes positive, keeps or restores its original data type, and is stored in the correct slot of the `output` array.
- Expected behavior: each output value should be positive and should match the original value's intended type.
- The utility code will mark each value in the output noting its type such as `[I]`, `[D]`, `[S]`, etc. This isn't something you need to add/change, but it should be used to determine if you solved it correctly.
- Workflow:
 - Add/commit the baseline code
 - Add UCID/date and planning comments before the solution code.
 - Commit the planning comments before committing the finished solution.
 - Test the PHP file locally before collecting evidence on `qa`.

Task #1 (2 pts.) – Scenario 3 Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show the relevant code with the UCID/date comment visible.
- Show the full browser output after running the file.

- Show the full browser output after running the file.

scenario 3 code

scenario 3 browser output

Part 2: URLs 100%

Details

- Paste the direct GitHub file link from the homework branch.
 - The GitHub URL must end in `.php`; zero credit for this URL if it does not.
- Paste the anticipated production URL for the PHP file.
 - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
 - The production URL must end in `.php`; zero credit for this URL if it does not.

URL #1

https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Module02-Homework/public_html/m02/hw/scenario3.php 🍏

URL #2

<https://sa3327-it202-450-m2026-prod.onrender.com/m02/hw/scenario3.php> 🍏

Part 3: Response 100%

Details

- Explain how your solution makes values positive.
- Explain how your solution handles type conversion.
- Name one type that needed extra attention.
- Do not restate the prompt.

Response

It loops through the array, it checks if its a string, if it is, it will use `preg_replace` to remove the "-". If not, it will just get the absolute value which will make it positive. These get pushed into the output array in their specific index.

Supports **bold**, *italic*, [\[links\]\(url\)](#), ``code``, etc.

245 chars

Section #4: (2 pts.) Scenario 4 Strings

100%

- File to edit and test: `public_html/m02/hw/scenario4.php`
- Goal: edit the `transformText()` function to clean and format text.
- Challenge 1: remove non-alphanumeric characters except spaces.
- Challenge 2: convert text to title case.
- Challenge 3: remove leading spaces, trailing spaces, and duplicate spaces between words.
- Note: Challenge 4 is extra credit item collected in another task.
- Expected behavior: assign the final phrase to `placeholderForModifiedPhrase`.
- Workflow:
 - Add/commit the baseline code
 - Add UCID/date and planning comments before the solution code.

Section #5: (+1 pt.) Scenario 4 Extra Credit Middle Characters

- File to edit and test: `public_html/m02/hw/scenario4.php`
- Goal: add the extra credit middle-character result to `transformText`.
- Challenge 4: extract up to the middle three characters from each cleaned phrase and assign the result to `placeholderForMiddleCharacters`.
- **Hint:** use the middle character +/- 1 (excluding the first and last characters) to find *up to* the middle 3 characters.
 - Examples: `short` should produce `hor`, `even` should produce `ve`, `eve` should produce `v`, and `abcdef` should produce `cde`.
- **Not enough characters rule:** assign `Not enough characters` when no middle characters remain after excluding the first and last character. For example, `a` should produce `Not enough characters`.
- URL note: this is part of Scenario 4, so it does not need a separate URL entry.
- Workflow:
 - Add UCID/date and planning comments before the solution code.
 - Commit the planning comments before committing the finished solution.
 - Test the PHP file locally before collecting evidence on `qa`.

🔓 Task #1 (+ 1 pt.) – Extra Credit Evidence

① Details

- Collect evidence for the extra credit solution.

🔓 Combo #1

🖼️ Part 1: Images 0%

① Details

- Show the relevant code with the UCID/date comment visible.
- Show the full browser output after running the file.

No images submitted yet.

✍️ Part 2: Response 0%

① Details

- Explain how your solution finds the middle of the string.
- Explain how your solution handles strings that are too short.
- Explain how your solution handles the different even/odd length phrases.
- Do not restate the prompt.

Response

Missing Response

Supports ****bold****, **italic**, [links](url), ``code``, etc.

0 chars

Section #6: (2 pts.) Misc

67%

🔓 Task #1 (0.67 pts.) – GitHub Details

100%

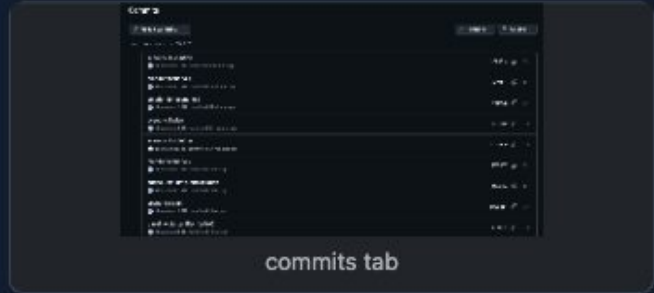
100%

①

Part 1: Images 100%

①

- Screenshot the Commits tab of the pull request.
- The screenshot should show at least the baseline commit, planning-comment commits, and solution commits.



commits tab

Part 2: URLs 100%

①

- Paste the pull request URL.
 - The URL must end in `/pull/#`, where `#` is the pull request number; zero credit for this URL if it does not.

100%

①

- Visit the WakaTime dashboard.
- Open **Projects**.
- Find your repository.
- Screenshot the overall time near the repository name (top of page).
- Screenshot the file-level activity (near the bottom).
- The purpose is evidence of activity; the duration isn't tied to the grade.
- The visual graphs are not required.

[illegible]



top of waka



files waka

≡ Task #3 (0.67 pts.) – Reflection

① Details

- Answer each question with at least a few reasonable sentences.

✎ Task #1 (0.22 pts.) – What did you learn during this assignment?

100%

Response

I learned PHP syntax and a lot of the built in methods.

Supports **bold**, *italic*, [links](url), `code`, etc.

55 chars

✎ Task #2 (0.22 pts.) – What was the easiest part of the assignment?

100%

Response

The easiest part was the first scenario, I've solved a lot of similar problems in previous classes.

Supports **bold**, *italic*, [links](url), `code`, etc.

99 chars

⌛ Saving...

✎ Task #3 (0.22 pts.) – What was the hardest part of the assignment?

100%

Response

The hardest part was scenario 4. Understanding the preg_replace function and researching other string functions was the most time consuming.

Supports **bold**, *italic*, [links](url), `code`, etc.

140 chars

⌛ Saving...

End of Assignment — make sure you click **Submit**.